

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 06 · LECTURE NOTES

Constraint Satisfaction

Search that knows about the problem's structure.

MODULE OUTCOMES

- Formulate a problem as a constraint satisfaction problem.
- Explain constraint propagation and arc consistency.
- Apply the standard variable and value ordering heuristics.
- Exploit problem structure, including tree decomposition.
- Apply constraint methods to procedural content generation.

1 The formulation

A CSP EXPOSES STRUCTURE THAT A GENERIC SEARCH HIDES

Variables, each with a domain of possible values, and constraints restricting which combinations are allowed. A solution is an assignment of every variable such that every constraint holds.

A generic search sees only opaque states and a successor function (Module 02 §1), **so it cannot know that the current state is doomed because of a conflict between two specific variables.** It discovers the failure only by exploring the subtree.

A CSP solver can see exactly that. So it prunes whole regions of the space from one violated constraint, and crucially **it can do so *before* search begins as well as during it (§2)** — which is the structural advantage over Module 02's black-box formulation.

And the goal is uniform across every CSP: every variable assigned, every constraint satisfied. **So the solver can be entirely generic and still exploit your problem's particular structure,** which is why industrial constraint solvers exist as general tools — unlike Module 02's searches, which must be written per problem.

Problem	Variables	Constraints
Sudoku	81 cells, each with domain 1–9.	All-different within each row, column, and 3×3 box.
Map colouring	One per region, domain the colours.	Adjacent regions take different colours.
Timetabling	For each class, a time slot and a room.	No two classes share a room and time; room capacity suffices; staff availability respected. The canonical industrial application.

Problem	Variables	Constraints
Level layout generation	For each tile position, which tile piece goes there.	Adjacent tiles' edges must match. See §4.
Puzzle generation	Which clues to include in a generated puzzle.	Exactly one solution exists — a constraint on the number of solutions, which is a genuinely useful and unusual thing to be able to express.

The last two rows are **procedural content generation**, and they are why this module belongs in a game-engines track — **Wave Function Collapse**, which generates a great deal of modern procedural level content, is a CSP solver (§4).

2 Constraint propagation

NODE CONSISTENCY remove values violating a unary constraint. Trivial; do it once at the start.

ARC CONSISTENCY for each constraint between X and Y, remove any value of X that has no supporting value in Y's domain. Repeat until nothing changes, because removals CASCADE -- shrinking X's domain may make a value of Z unsupported. This is AC-3, $O(e d^3)$, and it is the workhorse of constraint solving.

PATH / k-CONSISTENCY stronger inference, substantially more expensive, rarely worth computing in full.

WHAT AC-3 BUYS: on Sudoku, arc consistency alone solves easy puzzles with NO SEARCH AT ALL and reduces hard ones enormously. The pencil-marks technique human solvers use IS arc consistency, applied by hand.

MAINTAINING ARC CONSISTENCY DURING SEARCH (MAC) is the standard approach: assign a variable, propagate the consequences, and if any domain becomes empty, BACKTRACK IMMEDIATELY -- before exploring that subtree at all.

GLOBAL CONSTRAINTS MATTER ENORMOUSLY. "All-different over n variables" expressed as $n(n-1)/2$ pairwise inequalities propagates weakly: three variables each with domain {1,2} violates all-different but satisfies every pairwise constraint individually until two are assigned. A dedicated all-different propagator, using bipartite matching (CSCE 669 Module 11 section 1), detects it immediately. USE THE GLOBAL CONSTRAINT WHENEVER ONE EXISTS.

The global-constraint point is the most practically valuable thing in this module. The same logical condition expressed two ways prunes completely differently, which means modelling choice determines performance here exactly as it did in CSCE 669 Module 09 — and the lesson is the same: the formulation is worth more than the solver.

3 Ordering heuristics

- **Minimum remaining values (the most constrained variable).** Assign the variable with the smallest remaining domain next — so that if the assignment is going to fail, it fails high in the tree where the subtree being discarded is small. **Fail fast, and fail cheaply.**
- **The degree heuristic** as a tie-break: among variables with equally small domains, prefer the one involved in the most constraints on still-unassigned variables, since assigning it propagates furthest.
- **Least constraining value.** Among the values available for the chosen variable, try the one that rules out the fewest options for its neighbours — because you want to *succeed* on values, since any single success takes you forward.
- **That asymmetry is the thing to remember. Fail fast on variables; succeed fast on values. The two heuristics point in opposite directions and both are correct,** because you must try every value of a variable (so finding the failing variable early is good) and you need only one value to work (so finding the succeeding value early is good).
- **And conflict-directed backjumping** — on failure, jump back to the variable that actually participated in the conflict rather than the chronologically previous one, skipping intermediate assignments that were irrelevant. **These heuristics routinely change solve times by orders of magnitude and cost essentially nothing,** which makes them the best value available here.

STRUCTURE DECIDES TRACTABILITY

If the constraint graph is a tree, the CSP is solvable in $O(nd^2)$ — polynomial, by a single sweep that makes each node consistent with its parent, followed by a forward assignment pass. **No search at all,** which is a striking result given that CSP is NP-complete in general.

So look for tree structure, or create it. Cutset conditioning assigns values to a small set of variables chosen to break every cycle, then solves the remaining tree in polynomial time for each assignment of the cutset — so the cost is exponential only in the cutset size.

Tree decomposition generalises this properly: the cost is exponential in the *treewidth* of the constraint graph rather than in the number of variables — treewidth being, informally, how far the graph is from being a tree.

Which means a problem with a thousand variables and treewidth 4 is easy, and one with twenty variables and treewidth 19 is not. So measure the structure, not the size — and it is the same lesson as CSCE 669 Module 11 §4's checklist: whether a problem is tractable is a property of its structure, which is frequently a property of how you formulated it.

4 Constraints for content generation

WAVE FUNCTION COLLAPSE IS A CSP SOLVER WITH A TILESET

The variables are tile positions in a grid; the domains are the available tile pieces; the constraints are the adjacency rules stating which tile edges may meet which. That is the entire formulation.

The algorithm is minimum-remaining-values variable selection (§3), a weighted random choice among that variable's remaining values, then arc consistency propagation (§2). It is exactly this module's machinery with randomness added to the value choice — because a generator wants *variety* rather than one canonical solution, which is the only substantive difference from a solver.

So the designer authors *constraints* rather than content, and the solver produces unlimited valid content from them. That is the actual appeal, and it is a real shift in authoring: a tileset with adjacency rules is a compact specification of an infinite family of levels.

And the failure mode is contradiction — propagation empties some position's domain, so no valid completion exists from the current partial assignment. Backtrack, restart the affected region, or design the tileset so that contradictions cannot arise; the third is what shipping projects do, because it converts a runtime risk into a one-time authoring constraint.

- Express playability as a constraint rather than as a post-hoc check. 'A path exists from the entrance to the exit' can be a constraint (through a reachability propagator or a flow formulation), which eliminates the generate-and-reject loop entirely — and that loop is where naive procedural generation spends most of its time. The single most useful trick in this section.
- Author variety with weights on the value choice, so that common tiles appear commonly without being mandatory and rare ones appear occasionally — which is how a generated level gets a texture rather than a uniform distribution.
- Expect contradictions and decide the policy in advance: backtrack (correct, potentially slow), restart the affected region (fast, and can loop), or relax a soft constraint (always terminates, may produce something undesirable). Deciding this at design time rather than at 3 a.m. is worth the ten minutes.
- And use a real solver for hard instances. MiniZinc or OR-Tools CP-SAT will dispatch problems a hand-rolled backtracker cannot (CSCE 669 Module 13 §1), and they implement every global constraint in §2 properly.
- Then verify the output against the properties you wanted, because a constraint you forgot to write is not a constraint — and a generator will happily produce something technically valid and completely unplayable, which is CSCE 669 Module 12 §4's warning about optimisers exploiting your model.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Russell & Norvig — AIMA, chapter 6 (pseudocode free)	The reference for §1 through §3 , including AC-3 and the tree-structured result of §3.
Google OR-Tools — CP-SAT documentation (free)	A production constraint solver , with the global constraints of §2 and worked examples of every problem in §1's table.
Gumin — Wave Function Collapse (free, open source)	The §4 algorithm , with the original implementation and a large gallery of results.
Short & Adams (eds.) — Procedural Generation in Game Design	The §4 design perspective , by practitioners. Library copy; the free PCG Book at procedural-generation sites covers similar ground.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Formulate Sudoku as a CSP** and solve it with plain backtracking. Report node counts.
- 2 **Add arc consistency** and report the reduction. Confirm easy puzzles need no search.
- 3 **Express all-different pairwise and then as a global constraint**, and compare pruning on the same instance.
- 4 **Add minimum remaining values** and report the improvement.
- 5 **Add least constraining value** and report it separately.
- 6 **Construct a tree-structured CSP** and solve it without search.
- 7 **Compute the treewidth** of a constraint graph you built, approximately, and relate it to the solve time.
- 8 **Implement Wave Function Collapse** for a small tileset and generate twenty levels.
- 9 **Add a reachability constraint** and confirm every generated level is completable.
- 10 **Deliberately design a contradictory tileset** and implement each of the three failure policies.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does a CSP formulation expose that a generic search cannot?
- 2 Give five example CSPs with their variables and constraints.
- 3 Define arc consistency and say what human technique it corresponds to.
- 4 What is MAC, and why is it standard?
- 5 Why does a global constraint prune better than equivalent pairwise ones?

- 6 State the variable and value ordering heuristics and explain their asymmetry.
- 7 What is the complexity of a tree-structured CSP, and what generalises it?
- 8 Explain Wave Function Collapse as a CSP solver.
- 9 Why express playability as a constraint rather than a filter?